File Browser PRO

Native file browser for standalone



Documentation

Date: 08.06.2023

Version: 2023.2.0

© 2017-2023 **cross**tales LLC

https://www.crosstales.com

Table of Contents

1. Overview....3
2. Features....4
2.1. Filesystem operations....4
2.2. Documentation & control....4
2.3. Compatibility....5
3. Demonstration....6
3.1. DemoSync....6
3.2. DemoAsync....6
4. API....7
4.1. FileBrowser....7
4.1.1. Open single file....7
4.1.2. Open multiple files....7
4.1.3. Open single folder....7
4.1.4. Open multiple folders....8
4.1.5. Save file....8
4.1.6. Load data....8
4.1.7. Save data....9
4.1.8. Search for files....9
4.1.9. Get folders in a directory....9
4.1.10. Get drives of a device....10
4.1.11. Copy or move file....10
4.1.12. Copy or move folder....10
4.1.13. Show file location....10
4.1.14. Show folder location....10
4.1.15. Edit file with the default application....10
4.2. Callbacks....11
4.2.1. Open Files....11
4.2.2. Open Folder....11
4.2.3. Save File....11
4.3. UWP (WSA)....12
4.3.1. Settings....12
4.3.2. Files and folders....12
4.3.3. Search files and folders....12
4.3.4. Example....13
4.3.5. Reading and writing files....13
4.4. More details....13
4.5. Complete API....13
5. Third-party support (PlayMaker etc.)....13
6. Verify installation....14
7. Upgrade to new version....14
8. Important notes....14
8.1. Windows....14
8.2. MacOS....15
8.3. Linux....15
8.3.1. The following notes are only relevant for the older version of the library....15
9. Problems, improvements etc....16
10. Release notes....16
11. Credits....17
12. Contact and further information....18
13. Our other assets....19

Thank you for buying our asset "File Browser PRO"!

If you have any questions about this asset, send an email to fb@crosstales.com.

Please don't forget to rate it or write a little review – it would be very much appreciated.

1. Overview

File Browser is a wrapper for native file dialogs on Windows, macOS, Linux and UWP (WSA).

It also supports custom file browsers which allows it to be used on any platform!

Furthermore, it offers methods to load and save files, search for files and editing files with the default application of the operating system.

2. Features

2.1. Filesystem operations

- Open **file/folder**, **save file** dialogs supported
- **Multiple file** selection
- **Multiple folder** selection on **macOS** and **Linux**
- **Load** and **save** file data (incl. built-in support for images and text-files)
- **Search** for files
- Get **drives** for a device and **folders** for a location
- **Open** file or folder **location**
- **Copy** or **move** files and folders
- **Edit** file with the **default application**
- **Synchronous** and **asynchronous** (non-blocking) methods
- File extension **filters**
- Expand it via **custom file browsers**

2.2. Documentation & control

- **Test** all **dialogs** within the **editor**
- Powerful **API** for **maximum control**
- Detailed **demo scenes**
- Comprehensive **documentation** and **support**
- Full **source code** (including libraries)

2.3. Compatibility

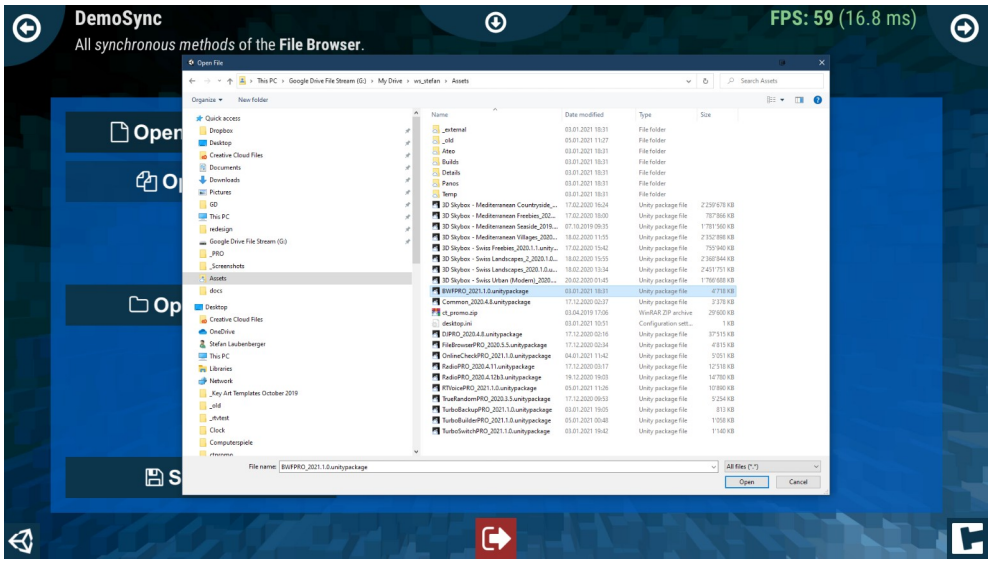
- Works **native** with **Windows**, **macOS**, **Linux** and **UWP (WSA)** in editor and runtime
- Support for **most platforms** via **Runtime File Browser**
- Support for **WebGL** via **WebGL Native File Browser**
- Compatible with **Unity 2019.4 – 2023**
- **C# delegates** and **Unity events**
- **PlayMaker actions**

3. Demonstration

The asset comes with two demo scenes to show the main usage.

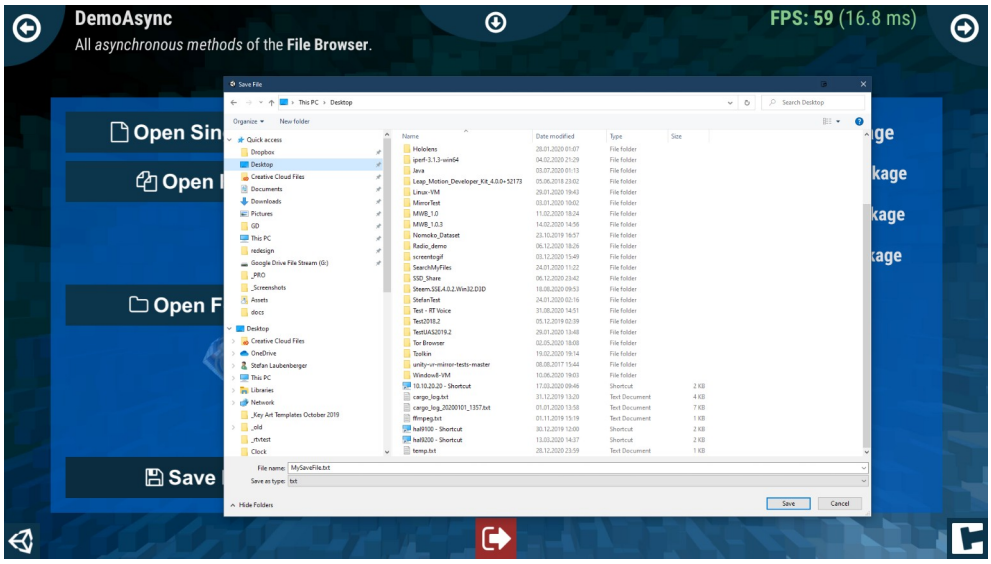
3.1. DemoSync

This scene shows open files/folders and a save dialogues.



3.2. DemoAsync

This scene shows asynchronous open files/folders and a save dialogues.



4. API

The asset contains various classes and methods. The most important ones are explained here.

Make sure to **include** the **name space** in the relevant source files:

```
using Crosstales.FB;
```

4.1. FileBrowser

The "FileBrowser.cs" is a singleton and contains the following important methods.

4.1.1. Open single file

```
public void OpenSingleFile() {
    string[] extensions = { "txt", "jpg", "pdf" };
    string path = FileBrowser.Instance.OpenSingleFile("Open file", "", "",
extensions);

    Debug.Log("Selected file: " + path);
}
```

4.1.2. Open multiple files

```
public void OpenFiles() {
    string[] extensions = { "txt", "jpg", "pdf" };
    string[] paths = FileBrowser.Instance.OpenFiles("Open file", "", "",
extensions);

    foreach (string path in paths)
    {
        Debug.Log("Selected file: " + path);
    }
}
```

4.1.3. Open single folder

```
public void OpenSingleFolder() {
    string path = FileBrowser.Instance.OpenSingleFolder();

    Debug.Log("Selected folder: " + path);
}
```

4.1.4. Open multiple folders

```
public void OpenFolders() {
    string[] paths = FileBrowser.Instance.OpenFolders();

    foreach (string path in paths)
    {
        Debug.Log("Selected folder: " + path);
    }
}
```

Note: this works does not work under Windows

4.1.5. Save file

```
public void SaveFile() {
    string[] extensions = { "txt" };
    string path = FileBrowser.Instance.SaveFile("Save file", "", "MySaveFile",
extensions);

    Debug.Log("Save file: " + path);
}
```

4.1.6. Load data

The get the data after calling "OpenSingleFile"- or "OpenFiles"-action use the following property:

```
//call "OpenSingleFile" or "OpenFiles"

byte[] data = FileBrowser.Instance.CurrentOpenSingleFileData;

//Typical use-cases
Texture2D tex = data.CTToTexture(); //returns a Texture of the data (supported
PNG and JPG)
Sprite sprite = data.CTToSprite(); //returns a Sprite of the data (supported
PNG and JPG)
string text = data.CTToString(); //returns a string of the data
```

4.1.7. Save data

To save the data of a "SaveFile"-action, set the following property before calling the action:

```
byte[] data; //set the data as byte-array before calling "SaveFile"

//Typical use-cases
Texture2D tex; //some texture
data = tex.CTToPNG(); //save texture as PNG
data = tex.CTToJPG(); //save texture as JPG
data = tex.CTToTGA(); //save texture as TGA
data = tex.CTToEXR(); //save texture as EXR

Sprite sprite; //some sprite
data = sprite.CTToPNG(); //save sprite as PNG
data = sprite.CTToJPG(); //save sprite as JPG
data = sprite.CTToTGA(); //save sprite as TGA
data = sprite.CTToEXR(); //save sprite as EXR

string text = "Hello there"; //some text
data = text.CTToByteArray(); //save string

FileBrowser.Instance.CurrentSaveFileData = data;

//call "SaveFile"
```

4.1.8. Search for files

```
string[] files = FileBrowser.Instance.GetFiles(<path>, false, "mp3"); //local
files = FileBrowser.Instance.GetFiles(<path>, true, "mp3"); //recursive
```

4.1.9. Get folders in a directory

```
string[] folders = FileBrowser.Instance.GetFolders(<path>); //local
folders = FileBrowser.Instance.GetFolders(<path>, true); //recursive
```

4.1.10. Get drives of a device

```
string[] drives = FileBrowser.Instance.GetDrives();
```

4.1.11. Copy or move file

```
FileBrowser.Instance.CopyFile(<pathToSource>, <pathToDest>); //copy
FileBrowser.Instance.CopyFile(<pathToSource>, <pathToDest>, true); //move
```

4.1.12. Copy or move folder

```
FileBrowser.Instance.CopyFolder(<pathToSource>, <pathToDest>); //copy
FileBrowser.Instance.CopyFolder(<pathToSource>, <pathToDest>, true); //move
```

4.1.13. Show file location

```
FileBrowser.Instance.ShowFile(<path>);
```

4.1.14. Show folder location

```
FileBrowser.Instance.ShowFolder(<path>);
```

4.1.15. Edit file with the default application

```
FileBrowser.Instance.OpenFile(<path>);
```

4.2. Callbacks

There are various callbacks available. Subscribe them in the "Start"-method and unsubscribe in "OnDestroy".

4.2.1. Open Files

```
OpenFilesStart();
OpenFilesStart OnOpenFilesStart;
```

Triggered whenever "OpenFiles" is started.

```
OpenFilesComplete(bool selected, string singleFile, string[] files);
OpenFilesComplete OnOpenFilesComplete;
```

Triggered whenever "OpenFiles" is completed.

4.2.2. Open Folder

```
OpenFoldersStart();
OpenFoldersStart OnOpenFoldersStart;
```

Triggered whenever "OpenFolders" is started.

```
OpenFoldersComplete(bool selected, string singleFolder, string[] folders);
OpenFoldersComplete OnOpenFoldersComplete;
```

Triggered whenever "OpenFolders" is completed.

4.2.3. Save File

```
SaveFileStart();
SaveFileStart OnSaveFileStart;
```

Triggered whenever "SaveFile" is started.

```
SaveFileComplete(bool selected, string file);
SaveFileComplete OnSaveFileComplete;
```

Triggered whenever "SaveFile" is completed.

4.3. UWP (WSA)

We designed File Browser to behave the same on all platforms.

Unfortunately, UWP (WSA) is very restrictive and the returned paths won't help to read or save any files, since access is only granted via StorageFile or StorageFolder.

Therefore, File Browser offers additional fields to use the selected files and folders, which are accessible via *FileBrowserWSAImpl*:

4.3.1. Settings

- **CurrentLocation**: path location for the file browser (default: PickerLocationId.ComputerFolder)
- **CurrentViewMode**: style of the file browser (default: PickerViewMode.List)

4.3.2. Files and folders

- **LastOpenFile**: Last file from the "OpenFiles"-dialog (StorageFile)
- **LastOpenFiles**: Last files from the "OpenFiles"-dialog (List<StorageFile>)
- **LastOpenFolder**: Last folder from the "OpenSingleFolder"-dialog (StorageFolder)
- **LastSaveFile**: Last file from the "SaveFile"-dialog (StorageFile)

4.3.3. Search files and folders

- **LastGetFiles**: Last files from the "GetFiles"-method (List<StorageFile>)
- **LastGetDirectories**: Last folders from the "GetDirectories"-method (List<StorageFolder>)
- **LastGetDrives**: Last drives from the "GetDrives"-method (List<StorageFolder>)

4.3.4. Example

```
#if (UNITY_WSA && !UNITY_EDITOR) && ENABLE_WINMD_SUPPORT
public async void ReadFile()
{
    FileBrowserWSAImpl.CurrentLocation = PickerLocationId.DocumentsLibrary;
    FileBrowserWSAImpl.CurrentViewMode = PickerViewMode.Thumbnail;

    FileBrowser.Instance.OpenSingleFile("txt");

    var file = FileBrowserWSAImpl.LastOpenFile;
    var lines = await FileIO.ReadLinesAsync(file);

    //do something with the content
}
#endif
```

4.3.5. Reading and writing files

Please follow this link to learn more about how-to read and save files under UWP:
https://docs.microsoft.com/en-us/windows/uwp/files/quickstart-reading-and-writing-files

4.4. More details

All methods can be found in "Example.cs" located under "Assets/Plugins/crosstales/FileBrowser/Demos"

4.5. Complete API

Please read the FileBrowser-api.pdf for more details.

5. Third-party support (PlayMaker etc.)

"File Browser" supports various assets from other publishers. Please import the desired packages from "Assets/Plugins/crosstales/FileBrowser/3rd party".

6. Verify installation

Check if File Browser is installed:

```
#if CT_FB
      Debug.Log("File Browser installed: " + Util.Constants.ASSET_VERSION);
#else
      Debug.LogWarning("File Browser NOT installed!");
#endif
```

7. Upgrade to new version

Follow this steps to upgrade the version of "File Browser":

1. Update "File Browser" to the latest version from the "Unity AssetStore"
2. Delete the "Assets/Plugins/crosstales/FileBrowser" folder from the Project-view
3. Import the latest version downloaded from the "Unity AssetStore"

8. Important notes

8.1. Windows

Windows allows setting the path for any application just once (for every dialog type). As soon as a user chooses a file or folder, it "remembers" the last destination and ignores any further attempts to change the path via code.

Multiple folder selection is not possible and the number of selectable files is limited to 256 with a maximal path length of 260 - if the path is shorter, more files can be selected. This setting can be changed in the Constants-class.

8.2. MacOS

Notarization and Mac App Store; to get an app through the Apples signing process, you may have to do one of the following things:

1) Add the following keys to the entitlement-file:

```
<key>com.apple.security.cs.disable-library-validation</key><true/>
<key>com.apple.security.files.user-selected.read-write</key><true/>
```

2) Sign the library after building:

```
codesign --deep --force --verify --verbose --timestamp --sign "Developer ID
Application : YourCompanyName (0123456789)"
"YourApp.app/Contents/Plugins/FileBrowser.bundle"
```

8.3. Linux

The library is tested under Ubuntu 22.04 with GTK3+.

Since there are so many different Linux distributions and configurations, we simply can't test and support them all.

Therefore, we included the whole source code; please follow the README.txt in the "FileBrowser - Linux (source).zip".

8.3.1. The following notes are only relevant for the older version of the library

The old library is tested under Ubuntu 18.04 with GTK3+ and X11.

To add support for Wayland, please create a shell-script that starts your application with X11 as backend, like this:

```
"GDK_BACKEND=x11 ./<applicationName>.x86_64" or "GDK_BACKEND=x11
./<applicationName>.x64"
```

9. Problems, improvements etc.

If you encounter any problems with this asset, just send us an email with a problem description and we will try to solve it.

We will try and make a version for all platforms as well, please bear with us.

10. Release notes

See "VERSIONS.txt" under "Assets/Plugins/crosstales/FileBrowser/Documentation" or online:

https://crosstales.com/media/data/assets/FileBrowser/VERSIONS.txt

11. Credits

The icons are based on Font Awesome.

Code partially based on:
https://github.com/gkngkc/UnityStandaloneFileBrowser

Improvements for the Linux version:
Yinon Oshrat (Intel)

Help fixing the Apple Silicion version:
Yuli Levtov (Volta)

12. Contact and further information

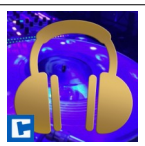
crosstales LLC

Schanzeneggstrasse 1

CH-8002 Zürich

Homepage:	https://www.crosstales.com/
Email:	fb@crosstales.com
AssetStore:	https://assetstore.unity.com/lists/crosstales-42213
Forum:	https://forum.unity.com/threads/file-browser-native-file-browser-for-windows-and-macos.510403/
Documentation:	https://www.crosstales.com/media/data/assets/FileBrowser/FileBrowser-doc.pdf
API:	https://www.crosstales.com/media/data/assets/FileBrowser/api/
Windows-Demo:	https://drive.google.com/file/d/1sE-6uhp2nk_5B85jvoiMWdk__HqUPSek/view?usp=sharing
Mac-Demo:	https://drive.google.com/file/d/1sAB953F-fpRmTSks9f2ZM0sMV7CEyyUA/view?usp=sharing
Linux-Demo:	https://drive.google.com/file/d/1LAm9v8Mu9jvF_8ZU0X3UU8nLKCdobzrj/view?usp=sharing
Android-Demo:	https://drive.google.com/file/d/139vmuauhc-prs_U868_W90x68cSyY1vj/view?usp=sharing

13. Our other assets

 **3D Skybox**	Those beautiful packages contain professional 8k, HDR, stereoscopic 360° real-world skyboxes for your projects.
 **Bad Word Filter**	The "Bad Word Filter" (aka profanity or obscenity filter) is exactly what the title suggests: a tool to filter swearwords and other "bad sentences".
 **DJ**	DJ is a player for external music-files. It allows a user to play his own sound inside any Unity-app. It can also read ID3-tags.
 **Online Check**	You need a reliable solution to check for **Internet availability**? Here it is!
 **Radio**	Radio allows implementing free music from Internet radio stations into your project..
 **RT-Voice**	RT-Voice uses the computer's (already implemented) TTS (text-to-speech) voices to turn the written lines into speech and dialogue at run-time! Therefore, all text in your game/app can be spoken out loud to the player.
 **True Random**	True Random can generate "true random" numbers for you and your application. The randomness comes from atmospheric noise, which for many purposes is better than the pseudo-random number algorithms typically used in computer programs.
 **Turbo Backup**	Turbo Backup is the fastest and safest way to backup your Unity project. It only stores the difference between the last backup, this makes it incredible fast.
 **Turbo Builder**	Turbo Builder creates builds for multiple platforms in one click. It works together with Turbo Switch to offer an incredible fast build pipeline.

 **Turbo Switch**	Turbo Switch is a Unity editor extension to reduce the time for assets to import during platform switches. We measured speed improvements up to 100x faster than the built-in switch in Unity.